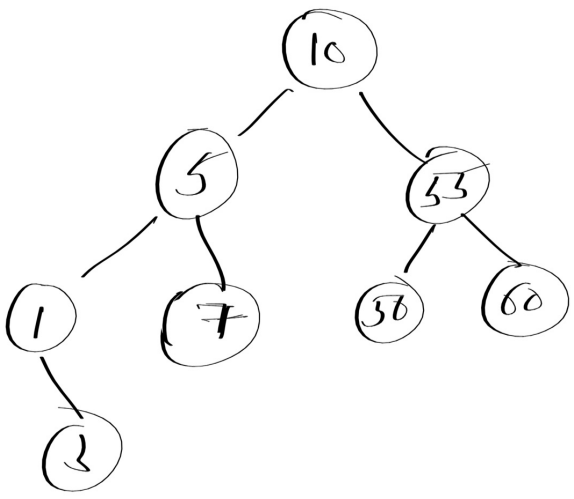
Day → 12

Tree Set → if we want in sorted set

- it use red black tree internally
- store unique elements
- null not allowed
- keep them sorted
- internally use Tree Map
  - uses Red-Black tree
    - Self balancing Binary Search Tree

Binary search tree → O(log n)

BST



Self balanced tree is if height of left sub tree and height of right sub tree should be -1 or 0 or 1.

So every treeSet's add, remove and other methods have Time Complexity $O(\log n)$

```
Set.add(10);
```

20, 1, 3, 25, 35

↓

1, 3, 10, 20, 25, 25

int values →

→ it compare its elements with each other. i.e the elements must be comparable

→ Java knows how to compare integer values & it has Comparable interface

So we have two methods / ways ⇒

(1) Comparable → for natural ordering

(2) Comparator → for custom ordering

Java SortedSet & NavigableSet

Detailed notes • interfaces, methods, examples, range views, outputs, and differences

1. Big picture: where do these interfaces fit?

SortedSet and NavigableSet are interfaces in the Java Collections Framework. They describe ordered sets: collections that keep unique elements in sorted order. TreeSet is a commonly used implementation.

```
Collection
  ■■■ Set
       ■■■ SortedSet
            ■■■ NavigableSet

TreeSet implements NavigableSet (and therefore SortedSet and Set).
```

Important: A TreeSet normally sorts elements in natural order (for example, integers in ascending order). A custom Comparator can change that order. Sets do not store duplicate elements.

2. Example set used throughout

```
NavigableSet<Integer> set = new TreeSet<>();
set.add(10);
set.add(20);
set.add(25);
set.add(30);
set.add(40);

System.out.println(set); // [10, 20, 25, 30, 40]
```

The elements are sorted automatically. You do not need to insert them in sorted order.

3. SortedSet interface

SortedSet extends Set and adds methods for working with elements in sorted order and taking range views. It provides ordering and range operations, but not the nearest-element navigation methods such as lower() and floor().

SortedSet methods

Method	What it does	Example / result for [10, 20, 25, 30, 40]
comparator()	Returns the Comparator used for ordering, or null when natural ordering is used.	set.comparator() → null for natural integer order
first()	Returns the lowest element in the set's ordering.	10
last()	Returns the highest element in the set's ordering.	40
headSet(toElement)	Returns a view of elements strictly before toElement; endpoint excluded.	headSet(25) → [10, 20]
tailSet(fromElement)	Returns a view from fromElement onward; endpoint included.	tailSet(25) → [25, 30, 40]
subSet(fromElement, toElement)	Returns a view from fromElement inclusive to toElement exclusive.	subSet(20, 40) → [20, 25, 30]
iterator()	Iterates in ascending set order (the order defined by the set).	10, 20, 25, 30, 40

The range methods return backed views, not independent copies. If the view permits a change, it is reflected in the original set. Adding an element outside the view's range through the view throws IllegalArgumentException.

SortedSet example

```
SortedSet<Integer> sorted = new TreeSet<>();
sorted.add(10);
sorted.add(20);
sorted.add(25);
sorted.add(30);
sorted.add(40);

System.out.println(sorted.first());             // 10
System.out.println(sorted.last());              // 40
System.out.println(sorted.headSet(25));         // [10, 20]
System.out.println(sorted.tailSet(25));         // [25, 30, 40]
System.out.println(sorted.subSet(20, 40));      // [20, 25, 30]
```

4. NavigableSet interface

NavigableSet extends SortedSet. It keeps all SortedSet capabilities and adds navigation methods that locate elements relative to a target, plus methods for descending order and inclusive/exclusive range boundaries.

Navigation methods: lower, floor, ceiling, higher

Method	Meaning	For target 25	Result
lower(e)	Greatest element strictly less than e (< e).	lower(25)	20
floor(e)	Greatest element less than or equal to e (≤ e).	floor(25)	25
ceiling(e)	Smallest element greater than or equal to e (≥ e).	ceiling(25)	25
higher(e)	Smallest element strictly greater than e (> e).	higher(25)	30

If no matching element exists, these methods return null. Remember: lower/higher exclude the target; floor/ceiling can include it.

First/last element and removal methods

Method	Meaning	Notes
first()	Returns first/lowest element in set order.	Throws NoSuchElementException if empty.
last()	Returns last/highest element in set order.	Throws NoSuchElementException if empty.
pollFirst()	Removes and returns first/lowest element.	Returns null if empty.
pollLast()	Removes and returns last/highest element.	Returns null if empty.

Descending-order methods

Method	What it returns	Example
descendingSet()	A backed view in reverse ordering.	→ [40, 30, 25, 20, 10]

Method	What it returns	Example
descendingIterator()	An iterator that visits elements in descending order.	40, 30, 25, 20, 10

NavigableSet range methods

Method	Boundary rules	Example / result
headSet(to, inclusive)	Elements before to; include to only if inclusive=true.	headSet(25, true) → [10, 20, 25]
tailSet(from, inclusive)	Elements after from; include from only if inclusive=true.	tailSet(25, false) → [30, 40]
subSet(from, fromInclusive, to, toInclusive)	Configures both endpoints independently.	subSet(20, true, 30, true) → [20, 25, 30]

The two-argument headSet/tailSet and four-argument subSet methods belong to NavigableSet. They are not declared by SortedSet, so the reference type must expose NavigableSet to call them.

NavigableSet complete example

```
NavigableSet<Integer> set = new TreeSet<>(
    Arrays.asList(10, 20, 25, 30, 40)
);

System.out.println(set.lower(25));        // 20
System.out.println(set.floor(25));        // 25
System.out.println(set.ceiling(26));      // 30
System.out.println(set.higher(25));       // 30
System.out.println(set.headSet(25, true));// [10, 20, 25]
System.out.println(set.tailSet(25, false));// [30, 40]
System.out.println(set.subSet(20, true, 30, true));
// [20, 25, 30]
System.out.println(set.descendingSet());  // [40, 30, 25, 20, 10]
```

For this example, add the import java.util.Arrays; if you use Arrays.asList(...). Alternatively, add the five values individually.

5. Difference between SortedSet and NavigableSet

Feature	SortedSet	NavigableSet
Inheritance	Extends Set.	Extends SortedSet.
Maintains sorted order	Yes.	Yes.
Unique elements	Yes, because it is a Set.	Yes, because it is a Set.
first() / last()	Yes.	Yes; inherited from SortedSet.
headSet(to) / tailSet(from) / subSet(from,to)	Yes; standard boundary rules.	Yes; inherited.
lower() / floor() / ceiling() / higher()	No.	Yes.
pollFirst() / pollLast()	No.	Yes.
Descending view / iterator	No.	Yes: descendingSet(), descendingIterator().
Inclusive/exclusive range options	Only standard defaults: head excludes end, tail includes start, subset includes start/excludes end.	Yes, via boolean overloads.

Feature	SortedSet	NavigableSet
Typical implementation	TreeSet can be referenced as SortedSet.	TreeSet can be referenced as NavigableSet.

A TreeSet object supports both interfaces, but the methods available through a variable depend on its declared reference type. For example, SortedSet s = new TreeSet<>(); does not let you call s.lower(25), even though the underlying TreeSet supports it.

6. Range view example: why adding 0 can fail

```
NavigableSet<Integer> set = new TreeSet<>();
set.add(-1);
set.add(0);
set.add(1);

SortedSet<Integer> view = set.subSet(-1, 0);
System.out.println(view); // [-1]

view.add(-1); // Allowed; duplicates are ignored
view.add(0);  // IllegalArgumentException: upper bound is exclusive
```

subSet(-1, 0) means -1 ≤ element < 0. The view contains -1 but not 0, and adding 0 through that view is not permitted. If you want both endpoints included, use set.subSet(-1, true, 0, true).

7. Quick revision sheet

Need	Method
Smallest / largest element without removing	first() / last()
Remove smallest / largest element	pollFirst() / pollLast()
Greatest value below target	lower(target)
Greatest value at or below target	floor(target)
Smallest value at or above target	ceiling(target)
Smallest value above target	higher(target)
Elements before target	headSet(target)
Elements from target onward	tailSet(target)
Elements in a half-open range	subSet(from, to)
Reverse-order view	descendingSet()

Memory trick

HEAD = elements before the boundary. **TAIL** = elements from the boundary onward. **LOWER** and **HIGHER** are strict. **FLOOR** and **CEILING** may equal the target.

8. Common mistakes to avoid

1. Calling headSet(25, true) on a variable declared as SortedSet: that overload is only available through NavigableSet.

2. Assuming subSet() creates a copy: it creates a backed view.

3. Forgetting the standard endpoints: SortedSet headSet excludes its endpoint; tailSet includes its start; subSet includes the start and excludes the end.

4. Assuming TreeSet always sorts numerically ascending: natural ordering is the default, but a Comparator can define another order.

5. Assuming null is returned by every method when the set is empty: first() and last() throw NoSuchElementException; pollFirst() and pollLast() return null.

End of notes